

Clinic Management System

Database Project Report

Fundamentals of Databases

Student Name	Farah Ghaly, Farah Ashraf
Student ID	221001666, 221001282
Course	Fundamentals of Databases

Table of Contents

1. Introduction
 2. Project Objective
 3. Requirements Covered
 4. Conceptual Design
 5. ER Diagram and Relationships
 6. Cardinality Ratios and Participation Constraints
 7. Relational Schema
 8. Table Constraints and Integrity Rules
 9. SQL Implementation
 10. Test Data
 11. Views
 12. Required Queries and Expected Outputs
 13. Triggers
 14. Bonus Flask Application
 15. How to Run the Project
 16. Conclusion
- Appendices: SQL Listings

1. Introduction

The Clinic Management System is a relational database project built to handle the day-to-day operations of a medical clinic. It keeps track of departments, physical clinic branches, doctors, patients, appointments,

diagnoses, appointment status, and payments. The design supports both appointment scheduling and long-term medical history tracking.

2. Project Objective

The goal is to design and implement a full MySQL database system for a clinic environment. The system lets the clinic manage departments, clinic branches, doctors, patients, and appointments in a structured, consistent way. It also includes views and queries for pulling medical and financial information when needed.

3. Requirements Covered

- Conceptual design with entities and attributes.
- ER diagram showing entities, relationships, cardinality ratios, and participation constraints.
- Relational schema with tables, attributes, primary keys, foreign keys, and constraints.
- DDL statements to create tables and views.
- DML statements to insert test data at least 10 records per main table.
- Three required SQL queries: fatty liver diagnosis, cardiology clinic addresses, and payment total for patient 12527.
- Triggers that block overlapping appointments for the same doctor.
- Bonus Flask application that reads from and writes to the MySQL database.

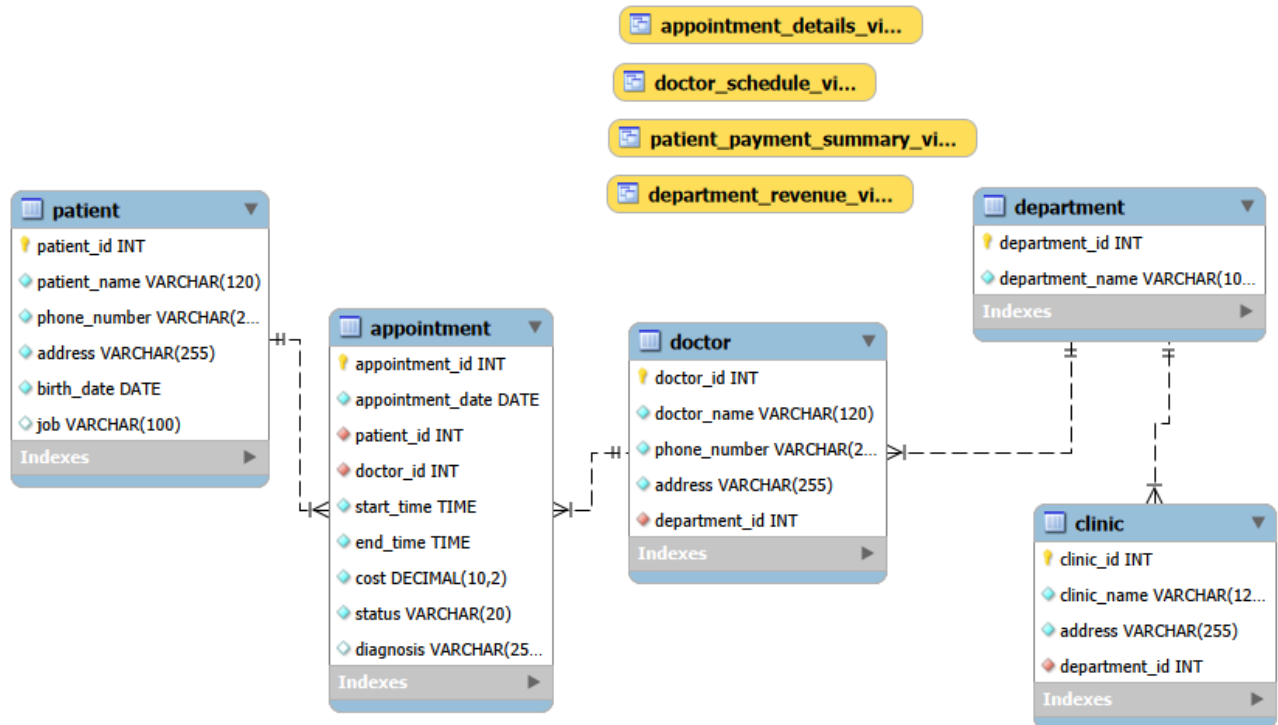
4. Conceptual Design

The database has five main entities: Department, Clinic, Doctor, Patient, and Appointment. Appointment is the central transactional table — it links a patient to a doctor at a specific date and time and stores the visit cost, current status, and diagnosis.

Entity	Attributes	Purpose
Department	department_id, department_name	A medical specialty such as Cardiology or Gastroenterology.
Clinic	clinic_id, clinic_name, address, department_id	A physical clinic branch belonging to a department.
Doctor	doctor_id, doctor_name, phone_number, address, department_id	A doctor assigned to one department.
Patient	patient_id, patient_name, phone_number, address, birth_date, job	A person receiving medical care.
Appointment	appointment_id, appointment_date, patient_id, doctor_id, start_time, end_time, cost, status, diagnosis	A visit booked between a patient and a doctor.

5. ER Diagram and Relationships

The EER diagram was generated directly from the MySQL schema. It shows the five base tables and the four database views. The views appear at the top of the diagram.



Appointment holds foreign keys to both Patient and Doctor. Doctor and Clinic each hold a foreign key to Department. This keeps department information in one place and avoids redundancy across tables.

6. Cardinality Ratios and Participation Constraints

Relationship	Cardinality	Participation	Explanation
Department → Clinic	1 : Many	Clinic total; Department partial	Every clinic must belong to a department. A department can exist before any clinics are added.
Department → Doctor	1 : Many	Doctor total; Department partial	Every doctor must belong to a department. A department can exist before any doctors are added.
Patient → Appointment	1 : Many	Appointment total; Patient partial	Every appointment must be linked to a patient. A patient can exist before booking any appointments.
Doctor → Appointment	1 : Many	Appointment total; Doctor partial	Every appointment must be assigned to a doctor. A doctor can exist before receiving any appointments.

7. Relational Schema

Department(department_id PK, department_name UNIQUE NOT NULL)

Clinic(clinic_id PK, clinic_name NOT NULL, address NOT NULL, department_id FK)

Doctor(doctor_id PK, doctor_name NOT NULL, phone_number UNIQUE NOT NULL, address NOT NULL, department_id FK)

Patient(patient_id PK, patient_name NOT NULL, phone_number UNIQUE NOT NULL, address NOT NULL, birth_date NOT NULL, job)

Appointment(appointment_id PK, appointment_date NOT NULL, patient_id FK, doctor_id FK, start_time NOT NULL, end_time NOT NULL, cost DECIMAL, status, diagnosis)

8. Table Constraints and Integrity Rules

Constraint Type	Implementation	Purpose
Primary Key	One per table.	Uniquely identifies every record.
Foreign Key	Clinic.department_id, Doctor.department_id, Appointment.patient_id, Appointment.doctor_id.	Maintains relationships between tables.
UNIQUE	Department names, doctor and patient phone numbers, clinic name/address pairs, doctor and patient time slots.	Prevents duplicates and scheduling conflicts.
CHECK	status values; cost >= 0; end_time > start_time.	Blocks invalid appointment data at the database level.
Trigger	BEFORE INSERT and BEFORE UPDATE on Appointment.	Catches partial time overlaps that a UNIQUE constraint can't detect.

9. SQL Implementation

The project uses MySQL. SQL files go in the /sql folder of the GitHub repository. The schema script creates the database, tables, constraints, and views. The data script loads test records. The trigger script handles scheduling integrity. The queries script contains the required and bonus queries.

File	Content
create_tables.sql	Creates the database, tables, constraints, and views.
load_data.sql	Inserts test data for Department, Clinic, Doctor, Patient, and Appointment.
triggers.sql	Triggers to block overlapping doctor appointments.
queries.sql	Required queries and additional sample queries.

10. Test Data

Test records were written to make sure the three required queries return real results — patient 12527 is included, there are Cardiology clinics, and there are fatty liver diagnoses within the past year.

Table	Records	Notes
Department	10	Includes Cardiology and other specialties.
Clinic	10	Includes Heart Care Clinic and Cardiology Follow-up Clinic.
Doctor	10	Each doctor is linked to a department.
Patient	10	Includes patient ID 12527.
Appointment	15	Covers all status types: scheduled, in progress, postponed, completed, and cancelled.

11. Views

View Name	Purpose
appointment_details_view	Joins appointments with patient names, doctor names, and department names.
patient_payment_summary_view	Shows appointment count and total amount paid per patient.
doctor_schedule_view	Shows each doctor's appointments and the associated patient names.
department_revenue_view	Calculates appointment count and total revenue per department.

12. Required Queries and Expected Outputs

12.1 — Patients diagnosed with fatty liver in the last year

```
SELECT DISTINCT p.patient_name
FROM Patient p
JOIN Appointment a ON p.patient_id = a.patient_id
WHERE LOWER(a.diagnosis) LIKE '%fatty liver%'
AND a.appointment_date >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR);
```

Expected Patient Names
Mostafa Mahmoud
Farah Ghaly

12.2 — Addresses of Cardiology clinics

```
SELECT c.clinic_name, c.address
```

```

FROM Clinic c
JOIN Department d ON c.department_id = d.department_id
WHERE d.department_name = 'Cardiology';

```

Clinic Name	Address
Heart Care Clinic	12 Nile Street, Dokki, Giza
Cardiology Follow-up Clinic	45 Tahrir Street, Downtown Cairo

12.3 — Total paid by patient ID 12527 in the last three years

```

SELECT p.patient_id, p.patient_name,
       SUM(a.cost) AS total_paid_last_three_years
FROM Patient p
JOIN Appointment a ON p.patient_id = a.patient_id
WHERE p.patient_id = 12527
      AND a.status <> 'cancelled'
      AND a.appointment_date >= DATE_SUB(CURDATE(), INTERVAL 3 YEAR)
GROUP BY p.patient_id, p.patient_name;

```

Patient ID	Patient Name	Total Paid (Last 3 Years)
12527	Mostafa Mahmoud	2100.00

13. Triggers

A UNIQUE constraint on (doctor_id, appointment_date, start_time) prevents two appointments with the same start time, but it doesn't catch partial overlaps like 10:00–10:30 and 10:15–10:45. The triggers below handle that by comparing the new time window against all existing windows for the same doctor on the same day.

```

USE clinic_management_system;
DELIMITER //
CREATE TRIGGER prevent_doctor_appointment_overlap_insert
BEFORE INSERT ON Appointment
FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT 1 FROM Appointment
        WHERE doctor_id = NEW.doctor_id
              AND appointment_date = NEW.appointment_date
              AND NEW.start_time < end_time
              AND NEW.end_time > start_time
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Doctor already has an overlapping appointment.';
    END IF;
END //

```

```

END IF;
END//
CREATE TRIGGER prevent_doctor_appointment_overlap_update
BEFORE UPDATE ON Appointment
FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT 1 FROM Appointment
        WHERE doctor_id = NEW.doctor_id
        AND appointment_date = NEW.appointment_date
        AND appointment_id <> NEW.appointment_id
        AND NEW.start_time < end_time
        AND NEW.end_time > start_time
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Doctor already has an overlapping appointment.';
    END IF;
END//
DELIMITER ;

```

14. Bonus Flask Application

The project includes a Flask app connected to MySQL via mysql-connector-python. It can read from and write to the database, and covers the main tables through a set of simple routes.

Route / Page	Function
/	Dashboard showing counts for departments, clinics, doctors, patients, and appointments.
/departments	Lists all departments.
/clinics	Lists clinics with department names.
/doctors	Lists doctors with department names.
/patients	Lists patients.
/appointments	Lists full appointment details.
/patients/add	Form to add a new patient.
/appointments/add	Form to add a new appointment.
/search	Search appointments by patient or doctor name.
/payments	Shows the total payment for a selected patient.

15. How to Run the Project

15.1 — Load the SQL files in MySQL

15.2 — Verify tables and record counts

```
USE clinic_management_system;
SHOW TABLES;
SELECT COUNT(*) AS departments_count FROM Department;
SELECT COUNT(*) AS clinics_count FROM Clinic;
SELECT COUNT(*) AS doctors_count FROM Doctor;
SELECT COUNT(*) AS patients_count FROM Patient;
SELECT COUNT(*) AS appointments_count FROM Appointment;
```

15.3 — Run the Flask app

```
cd src
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python app.py
```

Then open <http://127.0.0.1:5000> in the browser.

Clinic DB

Departments

Clinics

Doctors

Patients

Appointments

Add Patient

Add Appointment

Search

Payments

Clinic DB

Departments

Clinics

Doctors

Patients

Appointments

Add Patient

Add Appointment

Search

Payments

Clinic DB

Departments

Clinics

Doctors

Patients

Appointments

Add Patient

Add Appointment

Search

Payments

Clinic Management System

Database dashboard for departments, clinics, doctors, patients, and appointments.

Departments10

Clinics10

Doctors10

Patients10

Appointments15

Clinics

Clinic Id	Clinic Name	Address	Department Name
101	Heart Care Clinic	12 Nile Street, Dokki, Giza	Cardiology
102	Cardiology Follow-up Clinic	45 Tahrir Street, Downtown Cairo	Cardiology
103	Digestive Health Clinic	18 Abbas El Akkad Street, Nasr City, Cairo	Gastroenterology
104	Liver and Nutrition Clinic	22 El Nozha Street, Heliopolis, Cairo	Gastroenterology
105	Skin Care Clinic	9 Makram Ebeid Street, Nasr City, Cairo	Dermatology
106	Child Wellness Clinic	31 Syria Street, Mohandessin, Giza	Pediatrics
107	Bone and Joint Clinic	50 Gameat El Dewal Street, Mohandessin, Giza	Orthopedics
108	Brain and Nerve Clinic	77 El Merghany Street, Heliopolis, Cairo	Neurology
109	Vision Clinic	14 El Batal Ahmed Street, Mohandessin, Giza	Ophthalmology
110	Ear Nose Throat Clinic	60 Ramses Street, Cairo	ENT

Add Patient

Patient ID

Name

Phone

Address

Birth Date

Job

Add Patient

Clinic Management System | Page 10

Clinic DB

Departments

Clinics

Doctors

Patients

Appointments

Add Patient

Add Appointment

Search

Payments

Clinic DB

Departments

Clinics

Doctors

Patients

Appointments

Add Patient

Add Appointment

Search

Payments

Add Appointment

Appointment ID	Date	Patient
	mm/dd/yyyy	Dalia Fathy (12535)
Doctor	Start Time	End Time
Dr. Ahmed Hassan (201)	--:-- --	--:-- --
Cost	Status	Diagnosis
	Scheduled	
Add Appointment		

Patient Payment Summary

Dalia Fathy (12535)	Show Total
Patient: Dalia Fathy (12535) Appointments: 1 Total Paid: 500.00	

16. Conclusion

The Clinic Management System covers the full lifecycle of a relational database project: design, implementation, testing, and reporting. The schema handles departments, clinics, doctors, patients, and appointments cleanly, with foreign keys, check constraints, and triggers keeping the data consistent. The views and queries make it easy to pull useful information without writing complex joins every time. The bonus Flask app rounds things out with a working web interface on top of the database.